Pipeline de dados com a API TechTransfer da NASA

Guia passo a passo para replicar o projeto: coleta automática com GitHub Actions, banco no Supabase e painel no GitHub Pages

Grupo	11 (API: TechTransfer / Software)
Integrantes	Arthur, Helen e Mateus Rosler
Organização	https://github.com/TechTransfer-NASA
Repositório	https://github.com/TechTransfer-NASA/TechTransfer-Software
Painel	https://techtransfer-nasa.github.io/TechTransfer-Software/
Data	Novembro de 2026

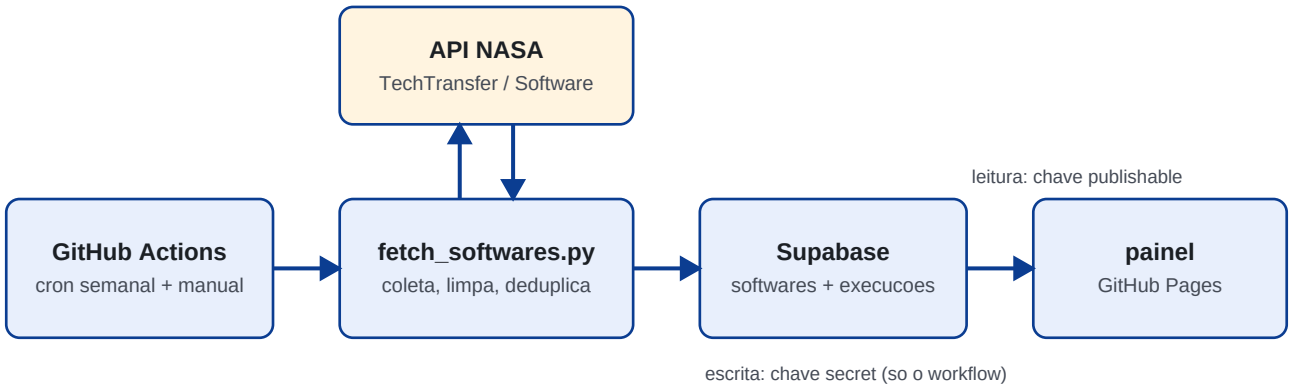
O que você vai construir

Um sistema que, toda semana e sem intervenção, consulta o catálogo de software da NASA, limpa os dados, grava em um banco PostgreSQL e os exibe em uma página web pública. Ao seguir este guia, uma pessoa sem contato com o projeto consegue reproduzi-lo do zero.

Sumário

- 1. Visão geral e arquitetura
- 2. O que você precisa antes de começar
- 3. Passo 1: GitHub (organização e repositório)
- 4. Passo 2: Supabase (projeto)
- 5. Passo 3: criar as tabelas
- 6. Passo 4: chaves e Secrets
- 7. Passo 5: a API e o script de coleta
- 8. Passo 6: o workflow
- 9. Passo 7: conferir os dados
- 10. Passo 8: o painel no GitHub Pages
- 11. Solução de problemas
- 12. Estrutura do repositório e referências

1. Visão geral e arquitetura



O fluxo tem quatro partes. O **workflow** do GitHub Actions roda toda segunda-feira (ou quando você clicar em Run workflow) e executa o **script Python**. O script consulta a **API da NASA**, trata os dados e os envia ao **Supabase**. O **painel** lê o Supabase pela API REST, sem passar pela NASA. Assim o site continua funcionando mesmo que a API da NASA esteja fora do ar, e fazemos poucas chamadas à NASA.

Por que um banco entre a NASA e o painel?

- **Independência:** o painel não depende da disponibilidade da NASA.
- **Dados limpos:** a API devolve listas de 13 posições, com HTML e campos vazios; o banco guarda campos nomeados e já tratados.
- **Histórico:** a tabela `execucoes` registra cada coleta, o que permite acompanhar falhas.

Componentes

Componente	Função	Arquivo / serviço
Workflow	Agenda e executa o pipeline	.github/workflows/update-data.yml
Script	Coleta, limpa, deduplica e grava	scripts/fetch_softwares.py
Banco	Tabelas, regras de acesso (RLS) e API REST	sql/setup.sql (Supabase)
Painel	Exibe os softwares e o histórico de execuções	pages/catalogo.html e pages/logs.html (GitHub Pages)

2. O que você precisa antes de começar

- Uma conta no **GitHub** (com permissão para criar uma organização).
- Uma conta no **Supabase** (plano gratuito basta).
- Opcional: uma chave gratuita da NASA em **api.nasa.gov** (Generate API Key). O endpoint usado funciona sem chave, mas o enunciado pede que ela seja guardada como Secret.
- Opcional, para testar no computador: **Python 3.10 ou superior** e a biblioteca `requests` (`pip install requests`).
- Um navegador. Não é necessário instalar nada para rodar o pipeline: ele executa nos servidores do GitHub.

Antes de criar qualquer coisa, defina os nomes. Neste projeto seguimos o padrão do enunciado, em que `N` é o número do grupo:

Item	Nome neste projeto
Organização (GitHub e Supabase)	SN-2026-GRUPO-N-NASA (ex.: SN-2026-GRUPO-11-NASA)
Repositório	TechTransfer-Software
Projeto Supabase	TechTransfer-Software-DB

3. Passo 1: GitHub (organização e repositório)

- Em github.com, clique em **+ > New organization** e escolha o plano gratuito. Use o nome definido na seção 2.
- Dentro da organização, **New repository**: nome `TechTransfer-Software`, visibilidade **Public**.
- Em **People > Invite member**, convide o professor (usuário `GilFerques`).
- Se já tinha uma organização ou repositório com outro nome: **Settings > General** permite renomear a organização e o repositório, e **Danger Zone > Transfer ownership** move um repositório de uma conta pessoal para a organização. O GitHub redireciona os links antigos, mas atualize o README.

Mais adiante você criará os arquivos do repositório (seção 12 mostra a estrutura). Pode enviá-los pela interface web (**Add file > Upload files**) ou pelo Git.

4. Passo 2: Supabase (projeto)

- Em supabase.com, crie uma **organização** com o mesmo nome da organização do GitHub.
- Dentro dela, **New project**: nome `TechTransfer-Software-DB`, região **South America (São Paulo)** e uma senha de banco (guarde-a).
- Em **Settings > Team > Invite**, convide o professor (rafael.ferques@ifpr.edu.br) com a permissão **Developer**.
- No plano gratuito há um limite de projetos ativos. Se aparecer o aviso, pause ou exclua um projeto antigo.

Projeto antigo com dados de outra atividade? Em vez de criar outro, você pode limpá-lo com o arquivo `sql/reset.sql` (apaga tabelas, views e funções do schema public; não mexe em Auth nem em Storage). O reset é irreversível: exporte antes, em Table Editor, o que precisar guardar.

5. Passo 3: criar as tabelas

No Supabase, abra **SQL Editor > New query**, cole o conteúdo **inteiro** do arquivo `sql/setup.sql` e clique em **Run**. O script pode ser executado mais de uma vez sem causar erro. Ele cria duas tabelas:

softwares: os dados da API

```
create table if not exists public.softwares (
  id bigint generated always as identity primary key,
  codigo text not null,              -- chave natural, ex.: GSC-16852-1
  titulo text, descricao text, categoria text,
  centro text, licenca text,
  urls text[] not null default '{}',  -- repositório / download
  ativo boolean not null default true,
  ultima_vez_visto timestamptz not null default now(),
  constraint softwares_codigo_key unique (codigo)  -- permite upsert sem duplicatas
);
-- (versão resumida: o arquivo sql/setup.sql tem todas as colunas)
```

A restrição **UNIQUE em** `codigo` é o que permite o *upsert*: se o software já existe, o banco atualiza a linha em vez de duplicá-la. O código é a chave natural porque é estável e legível (o campo de id interno da NASA é um texto opaco).

execucoes: o log do pipeline

Guarda uma linha por execução: `status` (`concluido`, `erro_parcial` ou `erro_critico`), registros processados, novos e inativados, lotes, erros, termos executados, uma mensagem e o detalhe de cada termo. O painel lê esta tabela para mostrar a última atualização.

Segurança: RLS e GRANTs

```
alter table public.softwares enable row level security;
create policy softwares_leitura_publica on public.softwares
  for select to anon, authenticated using (true);

grant select on public.softwares, public.execucoes to anon, authenticated;
grant all    on public.softwares, public.execucoes to service_role;
```

O **RLS** (Row Level Security) deixa as tabelas protegidas por padrão; a política libera apenas a **leitura** pública. Só a chave `service_role`, usada exclusivamente no workflow, pode escrever.

Depois de executar o script (e o primeiro workflow), o **Table Editor** mostra as duas tabelas preenchidas:

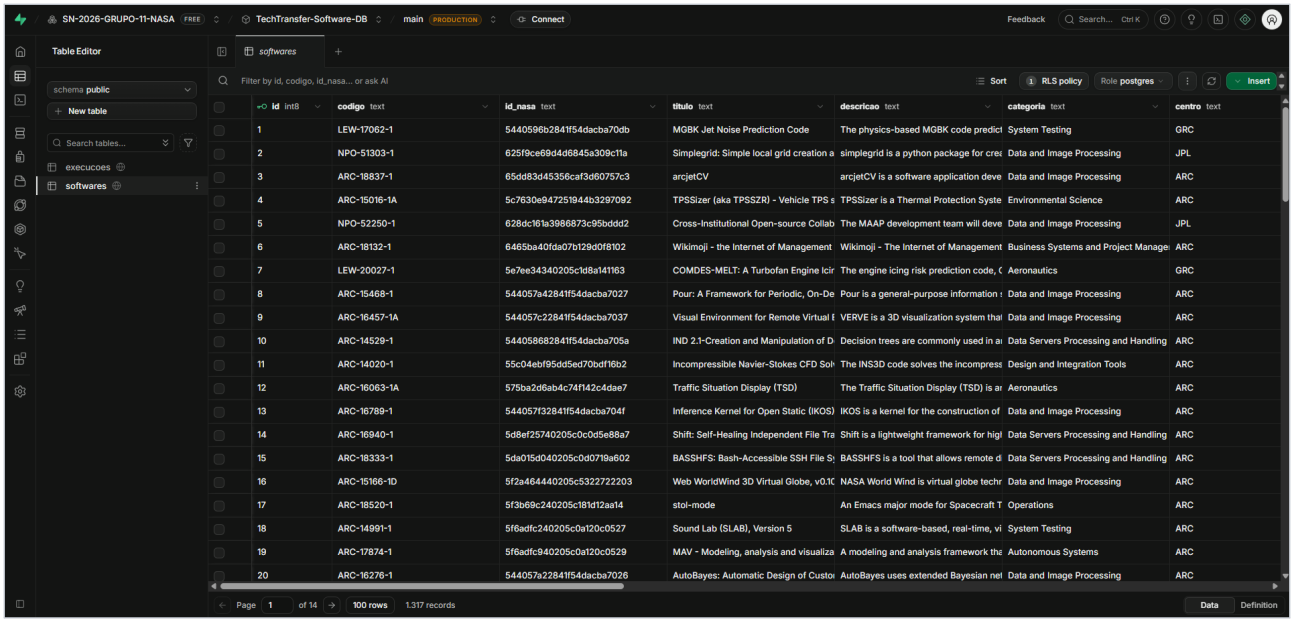


Figura 1: tabela softwares no Table Editor do Supabase (1.317 registros).

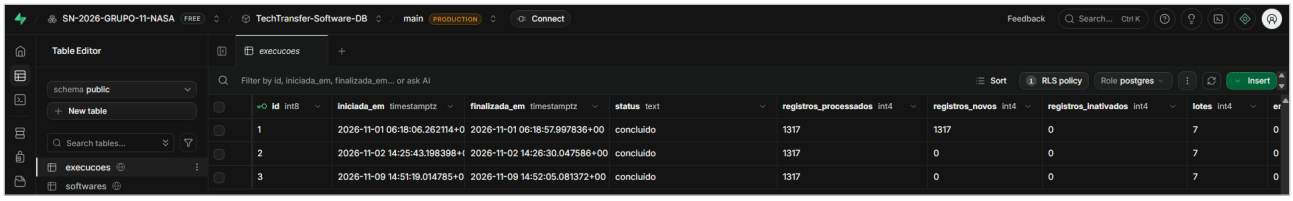


Figura 2: tabela execucoes, com uma linha para cada coleta (status concluido, 7 lotes, 0 erros).

6. Passo 4: chaves e Secrets

O workflow precisa de credenciais para gravar no Supabase. Elas ficam nos **Secrets** do GitHub e nunca no código.

Onde encontrar as chaves no Supabase

Em **Settings > API** (ou **API Keys**): a **Project URL**, a chave **publishable** (pública, somente leitura via RLS) e a chave **secret** (ou `service_role`, que escreve no banco e ignora o RLS).

Criar os Secrets no GitHub

No repositório: **Settings > Secrets and variables > Actions > New repository secret**.

Nome	Tipo	Valor
SUPABASE_URL	Secret	Project URL, ex.: https://abcdef.supabase.co
SUPABASE_SERVICE_KEY	Secret	chave secret / service_role do Supabase
NASA_API_KEY	Secret	chave gratuita de api.nasa.gov
NASA_API_BASE	Variable (opcional)	troca o endpoint da NASA; vazio usa o padrão do script

Atenção: a chave secret/service_role nunca pode aparecer no código do painel, em commits, prints ou mensagens. Se for exposta, gere outra imediatamente no Supabase e atualize o Secret. O painel usa somente a chave **publishable**.

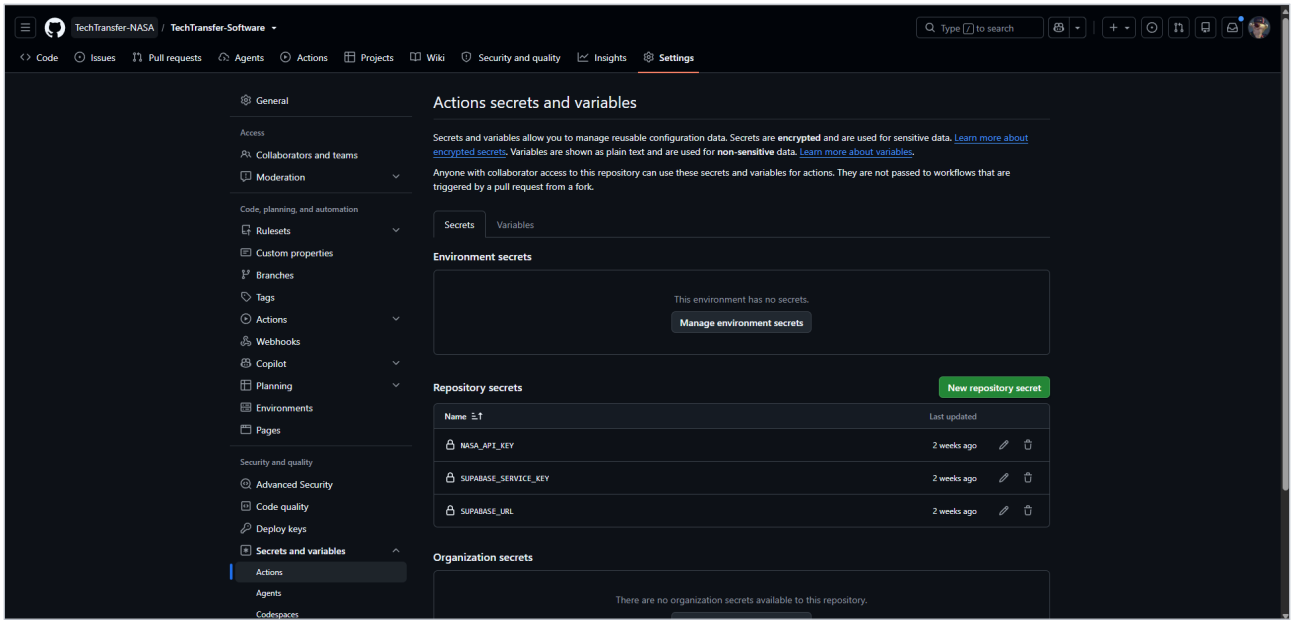


Figura 3: Secrets do repositório (aparecem só os nomes; os valores ficam criptografados).

7. Passo 5: a API e o script de coleta

O endpoint usado é `https://technology.nasa.gov/api/api/software/{termo}`, o endpoint do portal TechTransfer. O endereço documentado em api.nasa.gov/techtransfer/ redireciona para esse portal, e o grupo usou diretamente o endpoint dele. A API não é bem documentada, então medimos seu comportamento antes de escrever o script:

Característica	O que descobrimos
Formato	Cada item é uma lista de 13 posições, sem nomes de campos
Paginação	Não existe: tudo vem em uma chamada
Teto	Respostas são cortadas em 1000 itens; o campo total mostra o real
Busca	Por palavra inteira. Os códigos dos itens (GSC-, ARC-, NPO-...) também são pesquisáveis
Textos	Trazem marcação de destaque (span highlight), tags HTML e entidades como ®
Centros	A mesma sigla em grafias diferentes (LARC e LaRC)

Mapa das posições (software)

Posição	Campo	Posição	Campo
0	id interno da NASA	7	notas de acesso
1	código (chave natural)	8	links (podem ser vários, separados por ;)
2	título	9	centro da NASA
3	descrição	12	score da busca (descartado)
5	categoria	4, 10, 11	ignoradas (repetem o código ou vêm vazias)
6	licença / tipo de release		

O que o script faz

1. Busca cada um dos **32 termos** da lista (prefixos de código dos centros, como `arc`, `gsc` e `npo`, mais palavras comuns como `system` e `data`), com 1 s de pausa e até 3 tentativas por chamada.
2. **Valida e limpa** cada linha: exige 13 posições, remove HTML e destaques, decodifica entidades, converte textos vazios em NULL, padroniza o centro em maiúsculas e a categoria para o nome do site.
3. **Deduplica pelo código** antes de gravar. O mesmo software aparece em várias buscas, e enviar duas linhas iguais no mesmo lote causa o erro PostgreSQL 21000.
4. Faz **upsert em lotes de 200** linhas.
5. Marca como **inativos** os softwares não vistos há 21 dias (apenas em execuções sem erro).
6. Grava uma linha em `execucoes` e termina com erro (`sys.exit(1)`) se algo falhou, o que deixa o ícone do Actions vermelho.

Status da execução

Status	Quando acontece
concluido	Nenhum erro
erro_parcial	Chamada que falhou, resposta cortada em 1000, linha inválida ou lote que falhou
erro_critico	Coleta falhou na maior parte, veio vazia, ou trouxe menos da metade do que o banco já tem (a gravação é cancelada por segurança)

Testar no seu computador (sem gravar no banco)

```
pip install requests
python scripts/fetch_softwares.py --dry-run
```

O resumo deve mostrar algo em torno de 1,3 mil softwares únicos e `status previsto: concluido`. O número exato varia, pois a NASA atualiza o catálogo.

8. Passo 6: o workflow

O arquivo `.github/workflows/update-data.yml` agenda o script e injeta os Secrets como variáveis de ambiente:

```
name: Atualizar dados NASA TechTransfer Software
on:
  schedule:
    - cron: "0 6 * * 1"        # toda segunda, 06:00 UTC (03:00 em Brasilia)
  workflow_dispatch:            # execucao manual
permissions:
  contents: read
concurrency:
  group: update-data
  cancel-in-progress: false
jobs:
  update:
    runs-on: ubuntu-latest
    timeout-minutes: 20
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install requests
      - run: python scripts/fetch_softwares.py
        env:
          SUPABASE_URL: ${{ secrets.SUPABASE_URL }}
          SUPABASE_SERVICE_KEY: ${{ secrets.SUPABASE_SERVICE_KEY }}
          NASA_API_KEY: ${{ secrets.NASA_API_KEY }}
          NASA_API_BASE: ${{ vars.NASA_API_BASE }}
```

As três boas práticas exigidas: **permissions** mínimas (só leitura do repositório), **concurrency** (uma execução por vez) e **timeout-minutes** (a execução é cancelada se travar). Se você renomear o script, atualize o caminho na última etapa.

Para executar agora: aba **Actions**, escolha o workflow, **Run workflow**. A execução leva cerca de 1 a 2 minutos. Ícone verde significa que o script terminou com status concluido.

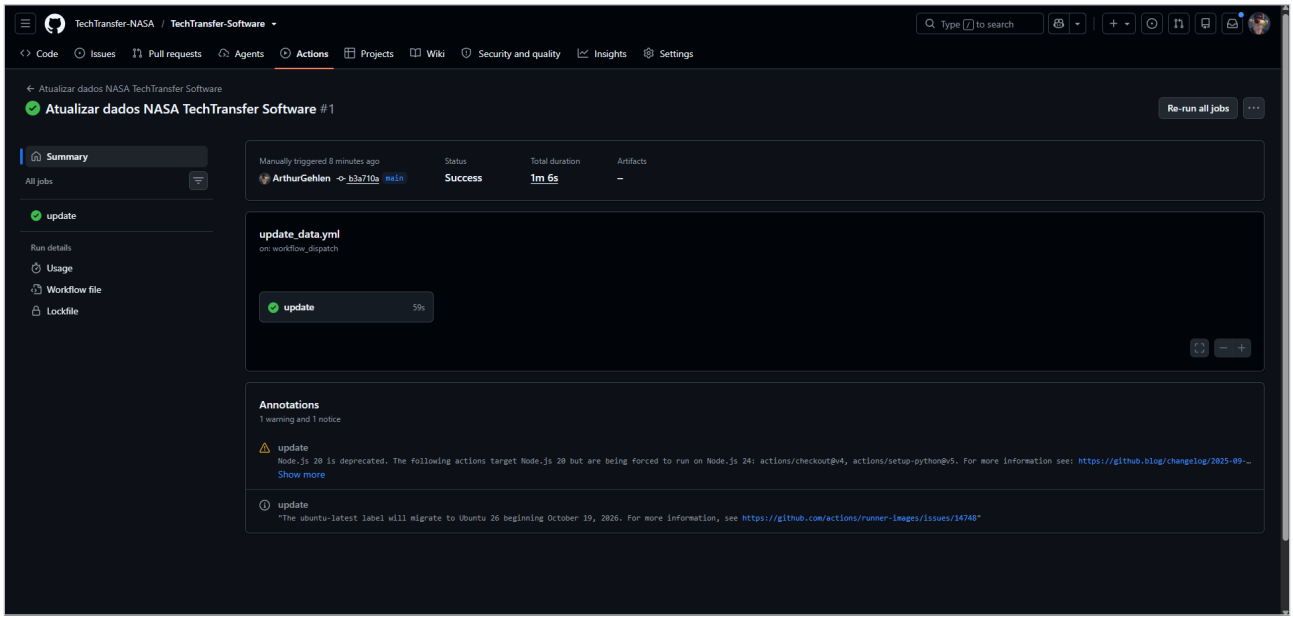


Figura 4: execução manual do workflow concluída com sucesso (1 min 6 s). Os avisos em Annotations são sobre a versão do Node.js usada pelas actions e não afetam o resultado.

9. Passo 7: conferir os dados

No SQL Editor do Supabase:

```
-- ultima execucao
select status, registros_processados, registros_novos, lotes, erros, mensagem
  from public.execucoes order by finalizada_em desc limit 1;

-- volume e variedade
select count(*) as total, count(*) filter (where ativo) as ativos,
       count(distinct centro) as centros, count(distinct categoria) as categorias
  from public.softwares;
```

Esperado: status `concluido`, `erros = 0`, cerca de 1,3 mil softwares, no máximo 12 centros e 15 categorias, e nenhum texto com `<span>` ou `'`.

A API REST pública

O Supabase gera uma API REST automática. Com a chave **publishable**, qualquer pessoa lê as tabelas liberadas pelo RLS. Abra no navegador (troque o projeto e a chave):

```
https://SEU_PROJETO.supabase.co/rest/v1/execucoes
  ?select=status,registros_processados,finalizada_em
  &order=finalizada_em.desc&limit=1&apikey=SUA_PUBLISHABLE_KEY
```

Deve retornar um JSON com a última execução. Se vier `[]` ou erro 401/403, revise o RLS, os GRANTs e a chave.

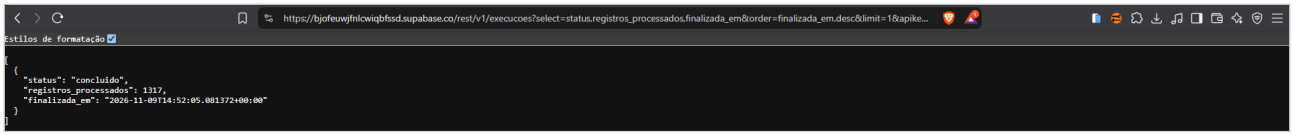


Figura 5: consulta à API REST no navegador, com a URL na barra de endereço e a resposta JSON da última execução.

Trocando `execucoes` por `softwares` (com `select=*&limit;=1`) a mesma API devolve um software, já com os campos tratados e o índice de busca:

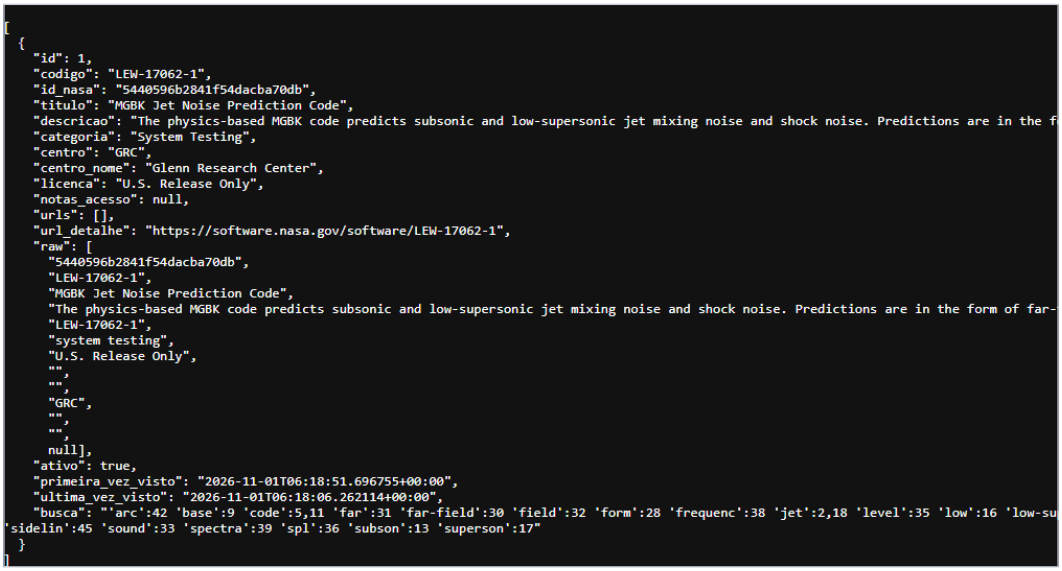


Figura 6: resposta da API REST para um software.

10. Passo 8: o painel no GitHub Pages

O painel é feito de páginas estáticas (`pages/`, `css/` e `js/`) que consultam a API REST do Supabase direto do navegador, sem servidor.

Publicar

- No repositório: **Settings > Pages**, em Source escolha **Deploy from a branch**, branch `main` e pasta `/(root)`.
- Após alguns minutos o endereço aparece na mesma tela, no formato `https://ORGANIZACAO.github.io/TechTransfer-Software/`.
- Use caminhos **relativos** nos arquivos (`../js/utils.js`, e não `/js/utils.js`), porque o site fica dentro de uma subpasta.
- A URL do projeto e a chave **publishable** ficam em `js/config.js`. Nunca coloque a chave secret ali.

O que o painel mostra

- **Catálogo:** busca por texto (em inglês) ou código, filtros por categoria, centro e licença, e cards com título, código, descrição (com botão *Ler mais*), etiquetas e link do repositório. Mostra o total (por exemplo, 10 de 1.317) e carrega mais itens sob demanda.
- **Logs:** o histórico das execuções do pipeline lido da tabela `execucoes`, com a data da última atualização, o status de cada coleta e o resultado por termo de busca.

Segurança no painel

- Somente a **publishable key** e a URL do projeto ficam no código. A secret nunca.
- Todo valor vindo do banco passa por `escapeHtml()` antes de entrar no HTML. As descrições da NASA podem conter aspas e símbolos como < e &, que quebrariam a página (ou executariam código) se fossem inseridos sem tratamento.
- Links vindos dos dados só são aceitos se começarem com http ou https.

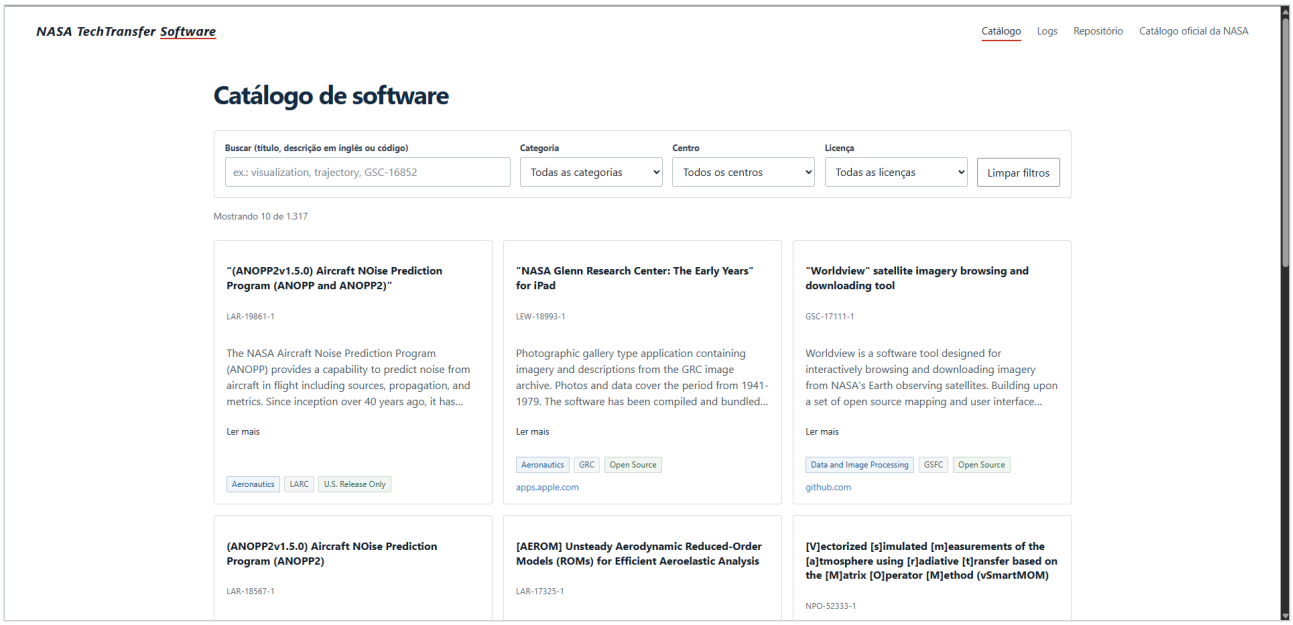


Figura 7: página do Catálogo, com busca, filtros, cards e botão Ler mais.

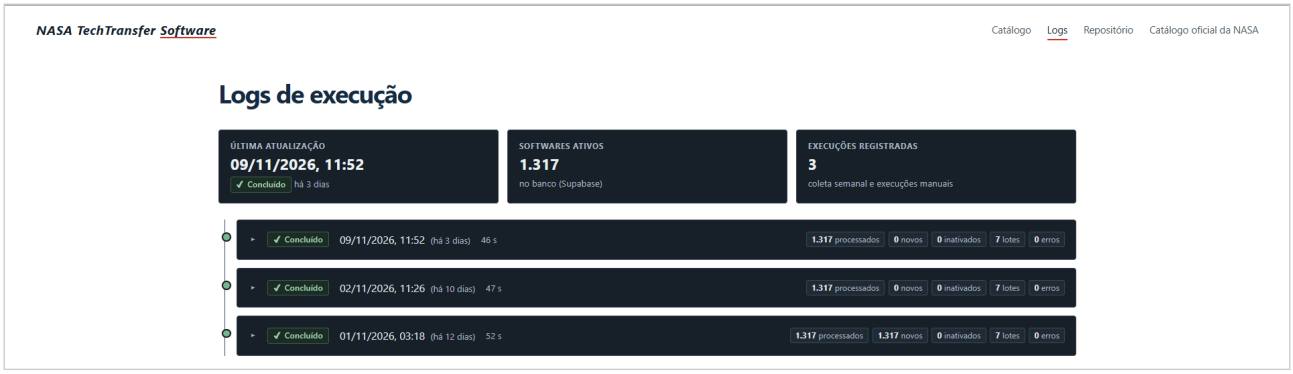


Figura 8: página de Logs, com o resumo da última atualização e o histórico das execuções do pipeline.

11. Solução de problemas

Sintoma	Causa provável e solução
Workflow não aparece em Actions	O arquivo precisa estar em .github/workflows/ na branch main.
defina SUPABASE_URL e SUPABASE_SERVICE_KEY	Nome do Secret diferente, ou Secret criado na organização e não no repositório.
HTTP 401 ou Invalid API key	A chave não é a secret/service_role, ou tem espaço no início ou no fim.
HTTP 404, relation softwares does not exist	O sql/setup.sql não foi executado neste projeto.
HTTP 403, permission denied	Faltaram os GRANTs para service_role: execute o setup.sql de novo.
Erro 21000 (ON CONFLICT ... a second time)	Linhas repetidas no mesmo lote. O script já deduplica por código; se editar o script, mantenha essa etapa.
Status erro_parcial	Veja a coluna mensagem e o JSON em detalhes (resultado por termo). Respostas cortadas em 1000 pedem termos mais específicos.
Status erro_critico: gravação cancelada por segurança	A coleta veio com menos da metade do que o banco tem: provável falha da API. Rode de novo mais tarde.
Painel: erro 400, column softwares.xxx does not exist	O nome da coluna na consulta não existe: a busca de texto usa a coluna busca.
Painel carrega mas sem dados	Chave ou URL errada no painel, ou falta a política de leitura (RLS) na tabela.
Painel abre sem estilo ou scripts	Caminhos começando com / no GitHub Pages: use caminhos relativos.

Para ver o detalhe de cada termo da última execução: `select jsonb_pretty(detalhes) from public.execucoes order by finalizada_em desc limit 1;`

12. Estrutura do repositório e referências

```
TechTransfer-Software/
|-- index.html                  entrada do site (GitHub Pages)
|-- README.md
|-- pages/                      catalogo.html, logs.html
|-- css/                        global.css, catalogo.css, logs.css
|-- js/                         config.js, utils.js, catalogo.js, logs.js
|-- scripts/
|   `-- fetch_softwares.py      coleta, deduplicacao e upsert
|-- sql/
|   `-- setup.sql               tabelas, constraint UNIQUE, RLS e GRANTs
|-- .github/workflows/
|   `-- update-data.yml         agendamento do pipeline
`-- docs/
    |-- tutorial.pdf  ai-interaction.md  reflexao.md  apresentacao.pdf
```

Referências

- Catálogo de software da NASA: software.nasa.gov
- Portal TechTransfer: technology.nasa.gov
- Chaves de API da NASA: api.nasa.gov
- Documentação do Supabase (RLS e API REST): supabase.com/docs
- Documentação do GitHub Actions e do GitHub Pages: docs.github.com